Implementing Stack using Java Collections

In Java, a Stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. This means the element that is inserted last is the first one to be removed. Think of a stack of plates: you add new plates to the top, and when you want a plate, you take it from the top.

While Java provides a `Stack` class in its `java.util` package, it's generally recommended to use other collection classes like `Deque` (specifically `ArrayDeque` or `LinkedList`) for implementing stack-like behavior due to `Stack` being a legacy class that extends `Vector` and is synchronized, which can lead to performance overhead in single-threaded environments.

Let's explore how to implement stack functionality using different Java Collection Framework classes.

I. Using the `Stack` Class (Legacy Approach)

The `java.util.Stack` class directly implements the LIFO principle. It extends `Vector`, which means it's a dynamic array and is synchronized.

Key Methods:

- `push(E item)`: Pushes an item onto the top of this stack.
- `pop()`: Removes the object at the top of this stack and returns that object.
- `peek()`: Looks at the object at the top of this stack without removing it.
- `empty()`: Tests if this stack is empty.
- `search(Object o)`: Returns the 1-based position where an object is on this stack. Returns -1 if the object is not found.

Example:

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        // Create a Stack of Strings
        Stack<String> stack = new Stack<>();

        // 1. Push elements onto the stack
        System.out.println("Pushing elements onto the stack:");
        stack.push("Apple");
        stack.push("Banana");
        stack.push("Cherry");
        System.out.println("Stack after pushes: " + stack); // Output: [Apple, Banana, Cherry]

        // 2. Peek at the top element
```

```
        System.out.println("\nPeeking at the top element: " + stack.peek()); // Output: Cherry
        System.out.println("Stack after peek: " + stack); // Stack remains unchanged: [Apple,
Banana, Cherry]

        // 3. Pop elements from the stack
        System.out.println("\nPopping elements from the stack:");
        System.out.println("Popped: " + stack.pop()); // Output: Cherry
        System.out.println("Stack after first pop: " + stack); // Output: [Apple, Banana]

        System.out.println("Popped: " + stack.pop()); // Output: Banana
        System.out.println("Stack after second pop: " + stack); // Output: [Apple]

        // 4. Check if the stack is empty
        System.out.println("\nIs stack empty? " + stack.empty()); // Output: false

        System.out.println("Popped: " + stack.pop()); // Output: Apple
        System.out.println("Stack after third pop: " + stack); // Output: []

        System.out.println("Is stack empty? " + stack.empty()); // Output: true

        // Trying to pop from an empty stack will throw EmptyStackException
        try {
            stack.pop();
        } catch (java.util.EmptyStackException e) {
            System.out.println("Caught EmptyStackException: " + e.getMessage());
        }
    }
}
```

Problem Definition: Max Stack

Problem Statement:
Design a stack that supports the following operations efficiently:

- push(x) — Push an element x onto the stack.
- pop() — Remove and return the top element.
- top() — Return the top element without removing it.
- getMax() — Return the maximum element in the stack in O(1) time.

Constraints:

- Only use built-in data structures (no external libraries).
- All operations must be performed in constant or amortized constant time.
- Must handle both positive and negative integers.

Sort a Stack

✅ Problem Definition

You are given a stack with unsorted integers.
You must sort the elements in ascending order (smallest at the top) using only one extra stack.

🚫 Constraints:

- ❌ You are not allowed to use recursion
- ❌ You are not allowed to use arrays or other data structures like queues or lists
- ✅ You can only use one additional stack

🎯 Objective

Transform:

Input Stack (Top → Bottom): 3, 5, 1, 4

Sorted Stack (Top → Bottom): 1, 3, 4, 5

🧠 Logic

1. Pop the top element curr from the originalStack
2. While the top of tempStack is greater than curr,
   → pop from tempStack and push it back to originalStack
3. Push curr into tempStack
4. Repeat until originalStack is empty
5. Finally, tempStack holds all elements in sorted order (smallest on top)
6. Transfer back if required

Next Greater Element (Using Stack)

✅ Problem Definition

Given an array of integers, your task is to **find the Next Greater Element (NGE)** for every element.

👉 The **Next Greater Element** for an element `x` is the first greater element on the **right side** of `x` in the array.
If there is no greater element, output `-1` for that position.

🧠 Concept

You have to solve this efficiently in **O(n)** time using a **stack**, not brute-force.

1. Create a stack.

2. Traverse the array from end to start:

   a. While stack is not empty and stack.peek() <= arr[i]:

      stack.pop(); // remove useless smaller elements

   b. If stack is empty:

      NGE[i] = -1

    Else:

      NGE[i] = stack.peek(); // next greater found

   c. Push arr[i] into the stack (it might be NGE for someone on the left)